

Manejo de Errores

Actividad 1 - Manejo de Errores: Tipos de Errores

Excepciones

Es un tipo especial de error que ocurre mientras el programa se esta ejecutando, por ej., dividir por cero, convertir un texto a numero y que no sea valido, acceder a un archivo que no existe, etc. Estas situaciones provocan errores predecibles, y con try/except es posible anticiparse y actuar de forma segura.

```
try:
    # Código que podría fallar
except TipoDeError:
    # Código que se ejecuta si ocurre ese error
```

- El bloque try contiene el código que puede lanzar una excepción.
- Si ocurre una excepción, Python salta al bloque except correspondiente.
- Si no ocurre ningún error, el bloque except no se ejecuta.

También se puede usar un except general sin especificar el tipo de error, aunque no es lo más recomendado:

```
try:
    # Código que puede fallar
except:
    print("Ocurrió un error")
```

El uso de try y except permite que los programas:

- No se detengan por errores controlables.
- Den mensajes amigables al usuario.
- Sean más robustos y confiables.

Tipos de excepciones comunes en Python (algunos de ellos)

- ZeroDivisionError: Se produce cuando se intenta dividir un número por cero.

```
try:
    resultado = 10 / 0 # Esto puede ser entrada del usuario
```

```
except ZeroDivisionError:
    print("No se puede dividir por cero.")
```

- ValueError: Ocurre cuando se intenta convertir datos a un tipo incompatible.

```
try:
    numero = int("hola")
except ValueError:
    print("Eso no es un número válido.")
```

- FileNotFoundError: Aparece cuando se intenta abrir un archivo que no existe.

```
try:
    archivo = open("inexistente.txt", "r")
except FileNotFoundError:
    print("El archivo no se encontró.")
```

- KeyError: Se lanza cuando se trata de acceder a una clave que no existe en un diccionario.

```
datos = {"nombre": "Ana"}

try:
    print(datos["edad"])
except KeyError:
    print("Esa clave no existe en el diccionario.")
```

- IndexError: Se genera al intentar acceder a una posición que no existe en una lista o tupla.

```
lista = [10, 20, 30]

try:
    print(lista[5])
except IndexError:
    print("Índice fuera de rango.")
```

- TypeError: Se lanza cuando se realizan operaciones entre tipos de datos incompatibles.

```
try:
    resultado = "5" + 2
except TypeError:
    print("No se pueden sumar cadenas y números.")
```

Actividad 2 - Excepciones en Python

Captura múltiple de excepciones

Python permite capturar múltiples excepciones utilizando varios bloques except si el programa lo necesita para validar distintos tipos de errores. Esto permite responder de forma distinta a distintos errores, lo cual mejora la robustez y claridad del programa.

```
try:
    numero = int(input("Ingrese un número: "))
    resultado = 10 / numero
except ValueError:
    print("Debe ingresar un número válido.")
except ZeroDivisionError:
    print("No se puede dividir por cero.")
```

Cada except puede manejar un tipo de excepción diferente. Python evalúa los bloques en orden, y ejecuta solo el primero que coincida con el error generado. Esto resulta útil cuando se quieren dar mensajes claros al usuario o tomar acciones diferentes dependiendo del tipo de programa.

Bloque finally

En Python existe un bloque especial llamado finally. Este bloque se ejecuta siempre, sin importar si ocurrió una excepción o no. El bloque finally se utiliza para realizar acciones de limpieza o tareas que deben ejecutarse pase lo que pase, como:

- Cerrar archivos abiertos.
- Cerrar conexiones a bases de datos.
- Liberar recursos ocupados.
- Mostrar un mensaje de salida.

```
try:
    archivo = open("datos.txt", "r")
except FileNotFoundError:
    print("Archivo no encontrado.")
    archivo = None
finally:
    if archivo:
        archivo.close()
    print("Este mensaje siempre se muestra. ")
```

- Si el archivo existe, se abre y luego se cierra normalmente.

- Si el archivo no existe, se muestra un mensaje y se evita el cierre de un archivo que no se pudo abrir.
- El bloque finally siempre se ejecuta, y en este caso cierra el archivo solo si fue abierto correctamente.

El bloque finally no reemplaza al except, sino que lo complementa. Se puede usar solo con try, o con try y except, dependiendo del caso.

* Importante: Aunque haya un return o una excepción en el bloque try o except, el código dentro de finally siempre se ejecuta antes de salir.

Microteaching

Tipos de errores

- Syntax Error: Errores en la estructura del código que impiden su ejecución.
- Name Error: Uso de variables o funciones no definidas.
- Type Error: Operaciones con tipos de datos incompatibles.
- Zero Division Error: Intentos de división por cero.
- Index Error: Acceso a índices fuera del rango de una lista.

Syntax Error: Error sintáctico

Ocurren cuando el código no sigue las reglas gramaticales de Python.

```
# Código incorrecto con SyntaxError
if True
    print(";Esto causará un error!")

# Python espera ':' después de la condición 'if'
# El error sería: SyntaxError: expected ':'
```

Name Error: Error de nombre

Aparece cuando se intenta usar una variable, función o módulo que no ha sido definido previamente.

```
# Código incorrecto con NameError
print(variable_inexistente)

# Python no encuentra 'variable_inexistente'
# El error sería: NameError: name 'variable_inexistente' is not defined
```

* Python es case-sensitive. Las variables edad, Edad y EDAD son diferentes.

Type Error: Error de tipo

Surgen cuando se intentan realizar operaciones con tipos de datos incompatibles.

- Operaciones inválidas: Intentar sumar un string con un entero

```
"5" + 3 # TypeError: can only concatenate str (not "int") to str
```

Zero Division Error

Ocurre cuando se intenta dividir un número por cero, una operación matemáticamente indefinida que Python no puede procesar.

```
10 / 0 # ZeroDivisionError: division by zero
```

Index Error: Error de índice

Aparece cuando se intenta acceder a una posición que no existe en una lista, tupla o string.

- Índices fuera de rango: Intentar acceder a una posición mayor que la longitud de la lista menos uno.

```
lista = [1, 2, 3]  
print(lista[5]) # IndexError: list index out of range
```

Estructura básica de try-except

El bloque try-except permite ejecutar código que puede fallar. Si ocurre un error dentro del bloque try, Python salta inmediatamente al bloque except correspondiente en lugar de detener el programa. Esta estructura da control total sobre cómo responder a diferentes tipos de errores.

Componentes del bloque try-except

- try: Contiene el código que se quiere ejecutar y que potencialmente puede generar errores.
- except: Define qué hacer cuando ocurre un error específico. Se pueden tener múltiples bloques except para diferentes tipos de errores.
- else: Se ejecuta solo si NO ocurrió ningún error en el bloque try. Es opcional pero útil para separar la lógica exitosa.
- finally: Se ejecuta SIEMPRE, haya o no error. Ideal para tareas de limpieza como cerrar archivos o conexiones.

Manejo efectivo de errores

- Ser específico con las excepciones: Capturar tipos de errores concretos en lugar de usar `except` genérico. Esto facilita la depuración y el mantenimiento.
- Mantener el bloque `try` pequeño: Solo incluye el código que realmente puede fallar. Esto hace más fácil identificar dónde ocurrió el problema.
- Proporcionar mensajes claros: Los mensajes de error deben ser informativos y orientar al usuario sobre qué salió mal y cómo solucionarlo.
- Documentar excepciones: Explicar porqué se está manejando ciertos errores y qué alternativas se ofrecen al usuario cuando ocurren.